

Weekly Report 2: Web-Based Real-Time Multiplayer Ludo Game

Date: July 20, 2026

Team Members:

- Shihab Uddin Patwary (ID: 2132685642)
- Md. Sajidul Islam (ID: 2121935642)
- Mohiuddin Sefat (ID: 2231193642)
- Sakiha Zaman (ID: 2233351642)
- Ishraq Hossain Rodro (ID: 2311675042)
- Musanna Ibn Amin (ID: 2233285042)

Executive Summary

This week, we transitioned our prototype into a component-driven client-server architecture. Building upon the foundational matrix and initial socket handshake from Week 1, we migrated the frontend to **Vite + React** with **Tailwind CSS** and engineered a centralized **Server-Authoritative Game Engine**. The system now successfully supports dynamic lobby creation, real-time turn management, server-side movement validation, and visual board integration.

Detailed Task Accomplishments

1. Interactive Visual Board & Frontend Architecture (/frontend)

- **React + Tailwind Migration:** Refactored static HTML assets into a modular Single Page Application (SPA) structured around clear route views: Home.jsx, Lobby.jsx, and Game.jsx.
- **Dynamic Grid Binding (Board.jsx & boardGeometry.js):** Successfully mapped the mathematical matrix into a responsive CSS Grid/Tailwind layout. Integrated coordinate handlers to dynamically project player tokens onto common tracks, safety zones, and home corridors based on backend state.
- **UI Game Controls (Dice.jsx & PlayerPanel.jsx):** Developed interactive components for animated dice rolling and real-time player status panels (active turn highlighting, player colors, and victory indicators).

2. Server-Authoritative Core Game Engine (backend/game/ludoEngine.js)

- **Rule Engine & State Machine:** Implemented full Ludo game logic on the backend to prevent client-side manipulation. Features include:

- Turn sequencing and dice roll evaluation.
- Validation of entry conditions (requiring a 6 to move out of base).
- Path navigation across 52 global common cells and color-coded home paths.
- Token collision detection, opponent capturing (sending captured tokens back to home base), and safe-zone enforcement.
- **Automated Unit Testing (backend/game/__test_engine.js):** Created a suite of headless tests to verify game loop transitions, boundary conditions, and win states prior to socket deployment.

3. Room Management & Duplex Socket Routing (backend/game/roomManager.js & sockets/index.js)

- **Isolated Room Allocation:** Developed roomManager.js to dynamically create, destroy, and isolate multi-player game instances using unique room codes.
- **Full-Duplex Event Lifecycle:** Expanded Socket.io handlers to manage real-time game flows:
 - CREATE_ROOM / JOIN_ROOM event handshakes.
 - ROLL_DICE broadcasts and server-calculated valid moves.
 - MOVE_TOKEN state sync across all connected clients in a room.
- **Graceful Disconnection:** Handled abrupt socket disconnects by notifying room participants and cleaning up abandoned session states.

4. API Endpoints & Auth Middleware Integration (backend/routes/auth.js & models/Match.js)

- **REST Authentication:** Finalized registration and login routes protected via JWT middleware (middleware/auth.js).
- **Match History Persistence:** Expanded Mongoose schemas (Match.js) to persist game outcomes, final rankings, and player statistics upon match completion.

Next Week's Objectives (Week 3)

1. **UX Polish & Motion Effects:** Add smooth token movement animations and visual feedback for captures and dice rolls on the frontend canvas.
2. **Audio & Reconnection Mechanics:** Implement sound effects for game actions and build reconnect logic so players can rejoin active sessions after brief network drops.
3. **Bot / AI Fallback & Final QA:** Introduce basic server-side AI bots to fill empty room slots and conduct end-to-end stress testing under multi-client network latency.